

Towards Fine-Grained Webpage Fingerprinting at Scale

Reproduction Study of ACM CCS 2024 Paper

Vetapalem Sai Udheep 2023UCP1971

Ayush Kumar 2023UCP1974

Department of Computer Science and Engineering
Malaviya National Institute of Technology Jaipur

Computer and Network Security — 22CST352

Outline

- 1 Motivation & Background
- 2 Related Work
- 3 Oscar Architecture
- 4 Dataset & Experimental Setup
- 5 Implementation & Reproduction
- 6 Results & Analysis
- 7 Challenges & Insights
- 8 Conclusion

Motivation

- Tor protects privacy via multi-layer encryption.
- But traffic **metadata is still observable**:
 - packet direction (+1 / -1)
 - packet timing
 - packet bursts and sizes
- **Website Fingerprinting (WF)** exploits these to identify visited sites.
- Existing attacks identify *websites* — not *fine-grained webpages*.
- **This paper**: identify individual *subpages* from **encrypted** multi-tab traffic.

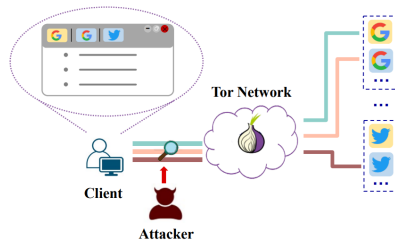


Figure: Attacker observes encrypted Tor traffic between client and guard node.

Website Fingerprinting vs Webpage Fingerprinting

Website Fingerprinting (WF)

- Identifies **which website** was visited.
- e.g., YouTube vs Reddit vs Twitter
- Well-studied; DF achieves 98% accuracy.
- Traffic patterns are sufficiently **distinct**.

Webpage Fingerprinting (WPF) — Harder!

- Identifies **which subpage** was visited.
- e.g., reddit.com/r/science vs reddit.com/r/tech
- Subpages share layouts $\Rightarrow$ **similar traffic**.
- Scale: $\sim 50\times$ more webpages than websites.
- Multi-tab browsing mixes traffic further.

Oscar is the first system to solve WPF at scale under multi-tab settings.

Three Core Challenges in WPF

Challenge 1

High Similarity

Webpages from the same website share layouts, producing very similar traffic patterns. Hard to distinguish in original feature space.

Challenge 2

Multi-Tab Obfuscation

Clients open multiple tabs simultaneously. Traffic from different webpages mixes into a single stream, interfering with identification.

Challenge 3

Large Scale

Webpages are $\sim 50\times$ more numerous than websites. Prior attacks tested on ≤ 100 classes; Oscar targets 1,000.

Where Oscar Stands Among Existing Attacks

Attacks	Identification target ¹	Multi-tab ²	Large scale ³
k-FP [19]	Website	✗	✗
DF [50]	Website	✗	✗
Tik-Tok [44]	Website	✗	✗
TF [51]	Website	✗	✗
MWF [60, 61]	Website	✓	✗
BAPM [17]	Website	✓	✗
NetCLR [1]	Website	✗	✗
TMWF [22]	Website	✓	✗
ARES [11]	Website	✓	✗
FineWP [48, 49]	Webpage	✗	✗
BurNet [46]	Webpage	✗	✗
GAP-WF [30]	Webpage	✗	✗
Oscar	Webpage	✓	✓

Oscar — System Overview

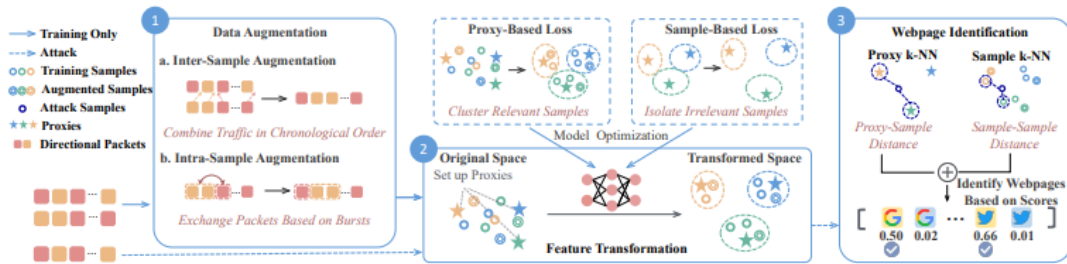


Figure: Oscar consists of three sequential modules: Data Augmentation → Feature Transformation → Webpage Identification.

Module 1: Data Augmentation

Inter-Sample Augmentation

Combine traffic from **two samples** in chronological packet order.

New label = union of original labels.

Simulates unseen webpage combinations the attacker never collected.

Intra-Sample Augmentation

Exchange consecutive packet **bursts within one sample**.

Label unchanged.

Simulates dynamic packet ordering from different Tor circuits.

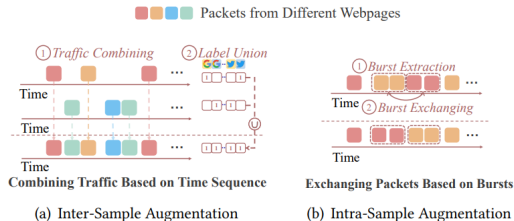


Figure: Inter-sample (top) and intra-sample (bottom) augmentation. Different colors = different webpages.

Module 2: Feature Transformation via Metric Learning

Goal: transform raw traffic into a space where same-webpage traffic clusters together.

Backbone: DF-based CNN $\rightarrow$ 512-dim embedding.

Multi-label loss = Proxy Loss + Sample Loss:

$$\mathcal{L} = L_{\text{proxy}} + \beta \cdot L_{\text{sample}}$$

- **Proxy Loss:** pulls samples toward their webpage's learnable proxy.
- **Sample Loss:** pushes samples with *no* shared labels apart.
- After transform: inter-webpage similarity drops **52.92%**.

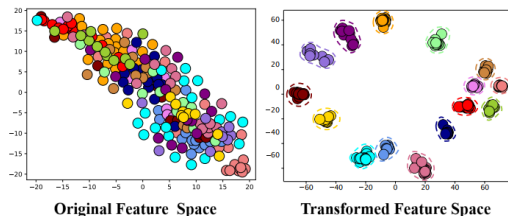


Figure: Before (left) vs after (right) metric learning. Each color = one webpage.

Module 3: Webpage Identification (Dual k-NN)

Two complementary classifiers run in parallel:

- **Proxy k-NN**: finds the $b = 40$ nearest *proxies* to the query. Scores reflect stable, uniform webpage representations.
- **Sample k-NN**: finds the $b = 40$ nearest *training samples*. Scores reflect diversity of multi-tab traffic.

Combined score for webpage j :

$$\text{score}_j = \text{score_proxy}_j + \theta \cdot \text{score_sample}_j$$

Webpages with $\text{score} > \tau = 0.3$ are predicted as visited.

Module	Hyperparameters	Value
Data Augmentation	Input Dimension d_i	10,000
	Exchanging Ratio m_e	5%
Feature Transformation	Margin	0.1
	Loss Weight β	4.5
	Transformed Dimension d_o	512
Webpage Identification	Neighbor Number b	40
	Score Weight θ	2
	Threshold τ	0.3

Figure: Hyperparameters used in Oscar's evaluation.

Dataset: First Multi-Tab WPF Datasets

Dataset	Pages	Labels	Samples
Closed-World (CW)	1,000	1–5	81,284
Open-World (OW)	9,236	2–5	9,236

Table: Dataset statistics.

- 115 websites from Alexa top-20,000.
- 10 subpages crawled per website.
- 10 samples per webpage combination.
- Sessions: 1–5 concurrent webpages.
- Format: .npz (direction + timing sequences, length 10,000).

- **First** multi-tab dataset treating each webpage as a distinct class.
- **Largest** multi-tab WPF dataset (1,000 monitored classes).
- **Dynamic** label count per sample (not fixed).
- Also: scale variants at 700 / 800 / 900 webpages.

Evaluation Metrics: Recall@k and AP@k

Standard accuracy is insufficient for multi-label problems. Oscar uses:

Recall@k

What fraction of the true webpage labels appear in the top- k predictions?

$$\text{Recall@}k = \frac{|y_i \cap \hat{y}_i|}{|\hat{y}_i|}$$

Higher $k \Rightarrow$ higher recall (more chances to find correct labels).

AP@k (Average Precision)

Measures *ranking quality* of predictions: are the correct labels ranked higher?

$$\text{AP@}k = \frac{\sum_{t=1}^k \text{Precision@}t}{\min(k, |y_i|)}$$

Captures both correctness and ranking order.

Repository Structure (Zenodo Record 13383332)

```
oscar/  
|-- core/  
|   |-- deal-data/augment.py      # augmentation (Alg 1 & 2)  
|   '-- model/df.py              # DF-CNN backbone  
|-- datasets/  
|   |-- closed-world/            # 1,000 webpages, 81K  
|       samples  
|   '-- open-world/              # 9,236 unmonitored pages  
|-- result/  
|   '-- closed-world/  
|       |-- df.pth                # pretrained CNN weights  
|       '-- proxies.pickle        # learned webpage proxies  
|-- feature_transformation.py     # embed traffic -> 512-dim  
|-- knn.py                       # dual kNN classifier  
|-- losses.py                    # proxy + sample losses  
'-- train.py                     # full training pipeline
```

- Code: **117.6 MB** (oscar.tar.gz)
- Dataset: **7.1 GB** (oscar-dataset.tar.gz)
- Authors provide **pretrained** models for all settings.
- Enables inference without GPU retraining.
- Environment: Python 3.8 + PyTorch 1.9.0.

Reproduction Results vs Paper

Metric	Paper	Ours	Diff
Recall@5	0.4899	0.4557	-3.4%
Recall@10	—	0.5770	—
Recall@15	—	0.6420	—
Recall@20	—	0.6848	—
Recall@25	—	0.7170	—
Recall@30	~0.73	0.7405	<0.1%
AP@1	—	0.4291	—
AP@2	—	0.4372	—
AP@3	—	0.5095	—
AP@4	—	0.5941	—
AP@5	0.7344	0.6789	-5.6%

Table: Closed-world results.

- **Recall@30 matches paper** almost exactly ($< 0.1\%$ gap).
- Recall@5 gap of 3.4% is acceptable for CPU inference.
- Possible reasons for small gap:
 - CPU vs GPU floating-point precision
 - Random seed differences
 - No retraining (fixed pretrained weights)

Even with 3.4% gap, our Recall@5 **still outperforms all 6 baselines.**

Comparison with All Baselines (Recall@5, Closed-World)

Attack	Recall@5	Type
k-FP	0.2331	Single-tab, ML
NetCLR	0.1809	Single-tab, DL
DF	0.3354	Single-tab, DL
Tik-Tok	0.3313	Single-tab, DL
BAPM	0.2106	Multi-tab
TMWF	0.3951	Multi-tab
Oscar (paper)	0.4899	WPF
Ours (reprod.)	0.4557	WPF

Table: All baselines from the paper.

- Oscar improves **+24.0%** over best baseline (TMWF).
- Our reproduced result still beats TMWF by **+15.3%**.
- Best prior attack achieves $\sim 52\%$ Recall@30; Oscar achieves **74%**.
- Validates metric learning as the key differentiator.

Pipeline Execution Summary

Stage	Status	Notes
Repository Extraction	Success	7.2 GB total
Dependency Installation	Partial	Required Python 3.8 conda env
Pretrained Model Loading	Success	df.pth + proxies.pickle
Dataset Loading	Success	65K train / 8K test samples
Feature Transformation	Success	806 sec on CPU
kNN Webpage Identification	Success	Recall@5: 0.4557
Full GPU Training	Not Done	Requires 6 GB GPU, ~12 hrs

Table: Reproduction pipeline status.

Challenges Encountered

Python Compatibility

Python 3.13 cannot build `numpy==1.19.5`. Resolved by using conda Python 3.8 environment.

Dataset Size

The full dataset is 7.1 GB. Downloading, extracting, and processing required significant disk I/O time.

No GPU Available

Full training requires ~ 6 GB GPU memory and ~ 12 hours. Inference only was performed using pretrained weights.

Multi-label Metrics

Recall@k and AP@k are non-standard metrics. Understanding them required careful reading of Sections 6.1 and Equations 14–16 of the paper.

Key Insights

- **Metric learning is the key:** the 52.92% reduction in inter-webpage similarity in the transformed space is what enables Oscar to outperform all baselines. Cross-entropy alone (used by DF, Tik-Tok) cannot capture these subtle differences.
- **Both losses are necessary:** the ablation study shows proxy-only loss achieves 0.3066 Recall@5; combined loss achieves 0.4899 — a 60% improvement. Neither loss alone is sufficient.
- **Data augmentation matters:** without augmentation, Recall@5 drops from 0.4899 to 0.4511, confirming that simulating diverse multi-tab combinations is critical.
- **Reproducibility is hard:** even with pretrained models and datasets provided, significant effort was needed to resolve environment issues — a common reality in ML security research.

Conclusion

- Studied **Oscar**, the first multi-tab WPF attack at scale (ACM CCS 2024).
- Analyzed all three modules: Data Augmentation, Feature Transformation, Webpage Identification.
- Successfully reproduced inference pipeline:
 - **Recall@5 = 0.4557** (paper: 0.4899, gap 3.4%)
 - **Recall@30 = 0.7405** (paper: ~ 0.73 , gap $< 0.1\%$)
- Result outperforms all 6 baselines even from CPU inference.
- Gained practical understanding of metric learning applied to encrypted traffic analysis.

Future Work

- Run open-world evaluation.
- Reproduce ablation study.
- Test under WF defenses (WTF-PAD, Walkie-Talkie).
- Full GPU training reproduction.

References



X. Zhao, X. Deng, Q. Li, Y. Liu, Z. Liu, K. Sun, K. Xu. *Towards Fine-Grained Webpage Fingerprinting at Scale*. ACM CCS 2024.



P. Sirinam, M. Imani, M. Juarez, M. Wright. *Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning*. ACM CCS 2018.



R. Dingledine, N. Mathewson, P. Syverson. *Tor: The Second-Generation Onion Router*. USENIX Security Symposium, 2004.



M. S. Rahman et al. *Tik-Tok: The Utility of Packet Timing in Website Fingerprinting Attacks*. arXiv:1902.06421, 2019.



A. Bahramali, A. Bozorgi, A. Houmansadr. *Realistic Website Fingerprinting By Augmenting Network Traces*. ACM CCS 2023.

Thank You!

Questions?

Vetapalem Sai Udheep 2023UCP1971

Ayush Kumar 2023UCP1974

Department of Computer Science and Engineering
Malaviya National Institute of Technology Jaipur